AI 计算系统
实验操作手册

实验序号：五

实验名称：基于 CANN 构建 VGG17

教　　师：朱光辉

学　　校：南京大学

时　　间：2022 年 8 月 23 日

五 基于CANN构建VGG17

5.1 实验目标

- 基于CANN高性能算子库构建VGG17网络
- 加载提供的权重（实验四得到的VGG17权重），编译graph得到离线模型，并利用实验三的测试样例对构建的模型进行推理验证，得到推理结果。

5.2 背景介绍

5.2.1 Tensor

Tensor是算子中的数据，包括输入数据与输出数据，TensorDesc类用于构建tensor对象，实例化tensor对象的shape、dtype和format。TensorDesc是对输入数据与输出数据的描述，TensorDesc数据结构包含的属性如下表所示：

属性	定义
名称(name)	用于对Tensor进行索引，不同Tensor的name需要保持唯一。
形状(shape)	Tensor的形状，例如 (10,)，(2, 3, 4)，默认值：无。形式：(i_1,i_2,..., i_n)
数据类型(dtype)	指定Tensor对象的数据类型，默认值：无。取值范围：float16，float32，int8，int16，int32，uint8，uint6，bool。
数据排布格式(format)	数据的物理排布格式。

5.2.2 数据排布格式

Format为数据的物理排布格式，定义了解读数据的维度，比如1D，2D，3D，4D，5D等。本次实验涉及到的数据排布格式为以下两种，同学们可在每个算子定义的函数内查看。

1. ND

ND支持原图中的所有数据排布格式。

2. NCHW和NHWC

在深度学习领域，多维数据通过多维数组存储，比如卷积神经网络的特征图（Feature Map）通常用四维数组保存，即4D，4D格式解释如下：

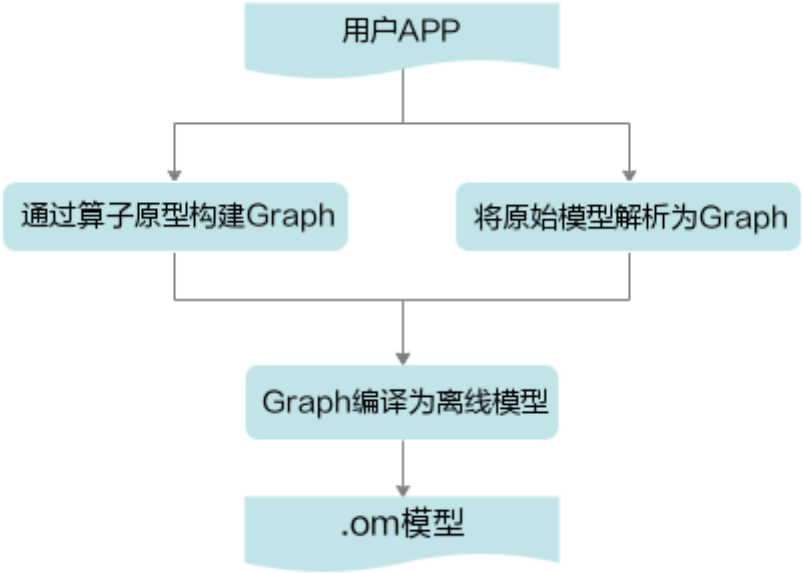
- N：Batch数量，例如图像的数目。
- H：Height，特征图高度，即垂直高度方向的像素个数。
- W：Width，特征图宽度，即水平宽度方向的像素个数。
- C：Channels，特征图通道，例如彩色RGB图像的Channels为3。

由于数据只能线性存储，因此这四个维度有对应的顺序。不同深度学习框架会按照不同的顺序存储特征图数据，比如Caffe，排列顺序为[Batch, Channels, Height,Width]，即NCHW。TensorFlow中，排列顺序为[Batch, Height, Width, Channels]， 即NHWC。

5.2.3 算子

深度学习算法由一个个计算单元组成，我们称这些计算单元为算子（Operator，简称 OP）。在网络模型中，算子对应层中的计算逻辑，例如：卷积层（Convolution Layer）是一个算子；全连接层（Fully-connected Layer， FC layer）中的权值求和过程，是一个算子。

而在模型的推理时，我们需要将网络模型解析为网络图（Graph）或者通过算子原型构造出Graph，Graph编译成离线模型，而后将推理数据送到网络中得到推理结果。



5.2.4 算子原型

算子原型定义了算子的数学含义，包含定义算子输入、输出、属性。

在开发阶段使用 REG_OP 宏，以 “.” 链接 INPUT、OUTPUT、ATTR 等接口注册算子的输入、输出和属性信息，最终以 *OP_END_FACTORY_REG* 接口结束，完成算子的定义。REG_OP 宏相当于定义一个该算子类型的类，后续开发者可以通过实例化该算子类来调用算子。

算子注册得到代码实现如下所示：

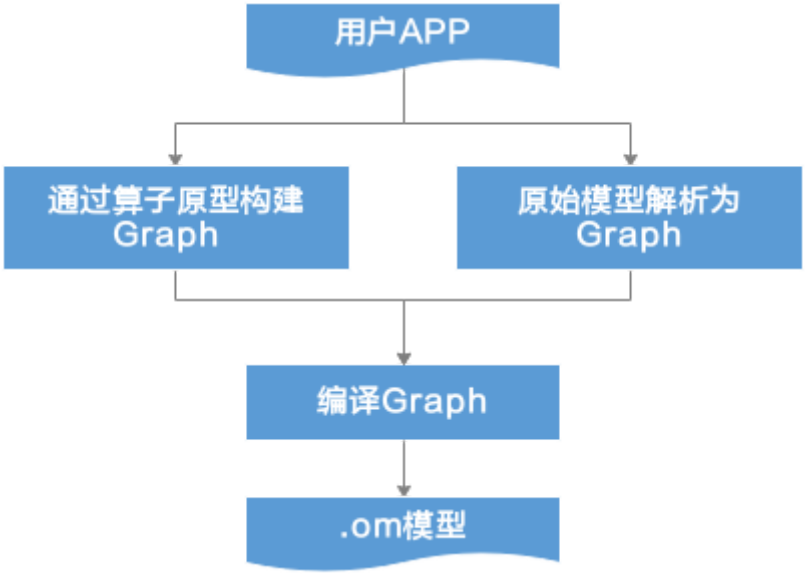
```
namespace ge{
 REG_OP(OpType) //算子类型名称
 .INPUT(x1, TensorType({ DT_FLOAT, DT_INT32 }))          //算子输入信息
 .INPUT(x2, TensorType({ DT_FLOAT, DT_INT32 }))          //算子输入信息
 .DYNAMIC_INPUT(x, TensorType{DT_FLOAT, DT_INT32})       //动态多输入场景下算子的输入
信息
 .OPTIONAL_INPUT(b, TensorType{DT_FLOAT})                //可选输入算子的输入信息
 .OUTPUT(y, TensorType({ DT_FLOAT, DT_INT32 }))          //算子的输出信息
 .DYNAMIC_OUTPUT(y, TensorType{DT_FLOAT, DT_INT32})      //动态多输入场景下算子的输出
信息
 .ATTR(x, Type, DefaultValue)                            //可选算子属性，当开发者不设
置算子对象属性时使用默认值
 .REQUIRED_ATTR(x, Type)                                 //必选算子属性，无默认值，开
发者必须设置算子对象的属性值
 .OP_END_FACTORY_REG(OpType) }
```

算子原型创建和代码详解请见CANN算子原型。

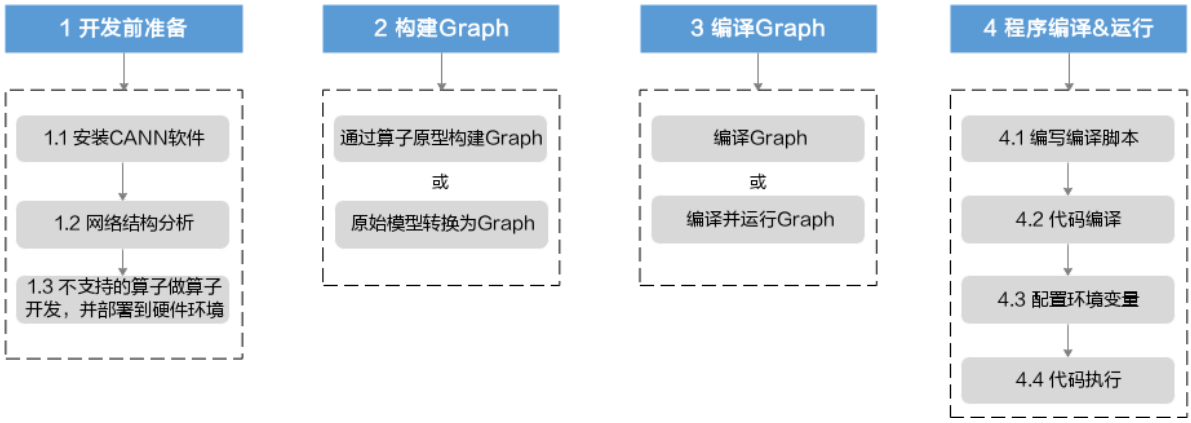
生成的算子信息会被注册到算子原型库中并自动生成对应的算子衍生接口。用户可通过这些接口在Graph中定义算子，然后创建一个Graph实例，并在Graph中设置输入算子、输出算子，从而完成Graph构建。

5.2.5 构建、编译和运行Graph的流程

用户可以通过开放的Ascend Graph编程接口，基于算子原型进行构图，并编译为离线模型，用于在昇腾AI处理器上进行离线推理。同时，也可以通过框架解析功能将主流的模型格式转换成CANN模型格式，从而隔离上层框架的差异，当前仅支持对Caffe/Tensorflow/ONNX原始框架模型的解析。



整体开发流程如下：



5.3 实验环境

- 操作系统：Ubuntu x86
- 编译器：g++
- 芯片：Ascend310
- python及依赖的库：python3.7.5
- 已完成昇腾AI软件栈在开发环境上的部署（CANN环境，需要完成驱动及CANN软件的安装）。

5.4 实验内容

本实验基于CANN高性能算子库，声明VGG17神经网络基本算子后加载所提供的的权重，接着构建VGG17网络模型，随后编译graph得到离线模型，并利用实验三的测试样例对构建的模型进行推理验证，得到推理结果。

本实验对基本算子已经提前声明，只需搭建VGG17神经网络并生成离线模型，利用实验三代码进行推理验证模型正确性。此实验重点是深入了解如何使用高性能算子搭建卷积神经网络。

5.5 实验步骤

本实验的代码目录介绍：

```
.
├── CMakeLists.txt                                          //编译脚本
├── data                                                    // 数据集文件夹
├── Makefile                                                //编译脚本
├── out                                                     //模型生成目录
├── scripts
|   ├── host_version.conf
|   └── testcase_300.sh                                     // 编译运行脚本
└── src
    └── main.cpp                                            // 实现文件
```

5.5.1 构建算子

在构建Graph之前，开发者需要了解算子模型，明确构建Graph所需的算子。

本实验不涉及自定义算子，因此对于已经在算子库注册了算子原型的算子，根据其注册名，从以下两种方式获取算子原型：

- 从算子清单中获得

  [Ascend310算子清单]

- 从“OPP安装目录/opp/op_proto/built-in/inc”算子原型定义的头文件中获取，例如：

  ```
  REG_OP(SoftmaxV2) //算子类型名称
      .INPUT(x, TensorType({DT_DOUBLE, DT_FLOAT16, DT_FLOAT}))
      .OUTPUT(y, TensorType({DT_DOUBLE, DT_FLOAT16, DT_FLOAT}))
      .ATTR(axes, ListInt, {-1})
      .OP_END_FACTORY_REG(SoftmaxV2)
  ```

5.5.1.1 Conv2D

Conv2D算子原型定义如下：

```
REG_OP(Conv2D)
    .INPUT(x, TensorType({DT_FLOAT16, DT_FLOAT, DT_INT8}))      //定义数据输入
    .INPUT(filter, TensorType({DT_FLOAT16, DT_FLOAT, DT_INT8})) //定义卷积核输入
    .OPTIONAL_INPUT(bias, TensorType({DT_FLOAT16, DT_FLOAT, DT_INT32})) //可选输入，卷积核的偏置bias
    .OPTIONAL_INPUT(offset_w, TensorType({DT_INT8}))  //可选输入，卷积核的offset_w，仍在算子清单中保留
    .OUTPUT(y, TensorType({DT_FLOAT16, DT_FLOAT, DT_INT32}))
    .REQUIRED_ATTR(strides, ListInt) //定义Conv2D的属性strides
    .REQUIRED_ATTR(pads, ListInt)    //定义Conv2D的属性pads
    .ATTR(dilations, ListInt, {1, 1, 1, 1}) //定义Conv2D的属性dilations
    .ATTR(groups, Int, 1)           //定义Conv2D属性group
    .ATTR(data_format, String, "NHWC") //定义Conv2D的数据输入格式
    .ATTR(offset_x, Int, 0)        //定义Conv2D的
    .OP_END_FACTORY_REG(Conv2D)    //结束算子注册
```

从Conv2D算子原型定义可以看到，Conv2D算子包括：

- 两个必选输入（INPUT）：x 和 filter
- 两个可选输入（OPTIONAL_INPUT）：bias 和 offset_w
- 两个必选属性（ATTR）：strides、pads
- 四个可选属性（REQUIRED_ATTR）：dilations、groups、data_format、offset_x

Conv2D算子定义的代码实现如下：

```
/*
>> 生成Conv2D算子：
    weight_shape:  用于定义卷积算子filter的权重shape
    conv_name：卷积算子的名称
    data：上一个算子。进行构图的连接
*/
Operator GenConv2dOp(Shape weight_shape,string conv_name,Operator data){

    //构造权重算子的描述信息desc_weight
    TensorDesc desc_weight(weight_shape,  FORMAT_NCHW, DT_FLOAT);
    //构造tensor
    Tensor weight_tensor(desc_weight);
    //计算出tensor需要的大小
    uint32_t weight_len = weight_shape.GetShapeSize() * sizeof(float);
    //从bin文件中加载数据，赋给tensor
    bool res = GetConstTensorFromBin(kPath+conv_name+".weight.bin",
weight_tensor, weight_len);
    if (!res) {
        cout << __LINE__ << "GetConstTensorFromBin Failed!" << endl;
    }
    //创建Const类型的权重算子，通过Const算子的属性value，传入tensor
    auto conv_weight = op::Const(conv_name+"_weight")
        .set_attr_value(weight_tensor);

    //创建卷积算子
    auto conv2d = op::Conv2D(conv_name)
        .set_input_x(data)               //定义输入，传入上一个算子
        .set_input_filter(conv_weight)   //定义卷积核，传入卷积核的权重
        .set_attr_strides({ 1, 1, 1, 1 }) //定义strides
        .set_attr_pads({ 1, 1, 1, 1 })    //定义pads
        .set_attr_dilations({ 1, 1, 1, 1 }) //定义dilations
        .set_attr_data_format("NCHW");     //定义输入数据的格式
```

```
    TensorDesc conv2d_input_desc_x(ge::Shape(), FORMAT_NCHW, DT_FLOAT);
    TensorDesc conv2d_input_desc_filter(ge::Shape(), FORMAT_NCHW, DT_FLOAT);
    TensorDesc conv2d_output_desc_y(ge::Shape(), FORMAT_NCHW, DT_FLOAT);
    conv2d.update_input_desc_x(conv2d_input_desc_x);       //更新卷积的输入信息
    conv2d.update_input_desc_filter(conv2d_input_desc_filter);//更新卷积的filter信息
    conv2d.update_output_desc_y(conv2d_output_desc_y);     //更新卷积的输出信息

    return conv2d;
}
```

1. 对于算子**必选输入和可选输入**：调用 `set_input_输入名称` 接口

```
.set_input_x(data)
.set_input_filter(conv_weight)
.set_input_bias(conv_bias)
```

data为整个graph的输入节点，通过Data算子构造，具体请参考[定义数据节点（Data）](#)。

conv_weight为常量数据，通过Const算子构造，具体请参考[定义数据节点（Const）](#)。

conv_bias为常量数据，通过Const算子构造，具体请参考[定义数据节点（Const）](#)。

2. 对于算子**必选属性和可选属性**：调用 `set_attr_属性名称` 接口

```
.set_attr_strides({1, 1, 1, 1})        //设置strides属性值
.set_attr_pads({1, 1, 1, 1})           //设置pads属性值
.set_attr_dilations({1, 1, 1, 1});     //设置dilations属性值
```

3. 对于**Conv2D等卷积类或对C轴处理敏感的算子**，建议通过 `update_input_desc_输入名称` 接口将Format信息设置为NCHW或者NHWC等，具体和用户需要处理的Format格式保持一致。

5.5.1.2 Data

Data算子是计算图数据输入节点，也是数据节点，算子原型定义如下：

```
REG_OP(Data)
 .INPUT(x, TensorType::ALL())          //定义输入信息，输入支持所有数据类型的输入
 .OUTPUT(y, TensorType::ALL())         //定义输出信息，输出支持所有数据类型的输出
 .ATTR(index, Int, 0)                  //定义算子属性
 .OP_END_FACTORY_REG(Data)             //结束算子注册
```

根据Data算子原型定义创建Data算子实例，实例代码如下。

定义Data类算子，名为data1，定义data1算子的信息描述为desc_data，desc_data定义了输入数据的shape。data1通过 *“update_input_* desc输入名称 ” 和 “ *update_output_desc*输出名称” 接口设置算子的Shape、Format和Dtype，更新算子的信息描述。

```
//实例化Data算子的输入数据的描述信息
auto shape_data = vector<int64_t>({1,3,224,224}); //输入数据[N,C,W,H],推理时batchsize为1
TensorDesc desc_data(ge::Shape(shape_data), FORMAT_NCHW, DT_FLOAT); //实例化一个算子信息描述的TensorDesc类，定义传入数据的shape，并将shape传入desc_data

// 实例化Data算子，名为data1
auto data = op::Data("data1");
data.update_input_desc_x(desc_data);  //更新data1算子的输入数据信息描述，定义输入数据的shape，format和dtype
data.update_output_desc_y(desc_data); //更新data1算子的输出数据信息描述，定义输入数据的shape，format和dtype
```

5.5.1.3 Const

权值、偏置等信息为常量Tensor，可以使用Const算子实现。Const算子定义如下：

```
REG_OP(Const)
    .OUTPUT(y, TensorType({DT_FLOAT, DT_FLOAT16, DT_INT8, DT_INT16, DT_UINT16, \
        DT_UINT8, DT_INT32, DT_INT64, DT_UINT32, DT_UINT64, DT_BOOL,
DT_DOUBLE})) //定义算子的输出，支持如上类型
    .ATTR(value, Tensor, Tensor())  //定义算子的属性，value表示常量算子对应的值
    .OP_END_FACTORY_REG(Const)      //结束算子定义
```

Const算子一般从bin文件中读取权重，偏置等数据，构造权重算子的代码实现如下：

```
//构造权重算子的描述信息desc_weight_1
auto weight_shape = ge::Shape({ 5,17,1,1 });                  //实例化一个Shape类
TensorDesc desc_weight_1(weight_shape, FORMAT_NCHW, DT_INT8); //实例化一个算子信息描述的TensorDesc类，得到权重的描述信息desc_weight_1

//从bin文件中加载数据，构造tensor，为了将tensor传入权重算子里
Tensor weight_tensor(desc_weight_1);                          //构造tensor
uint32_t weight_1_len = weight_shape.GetShapeSize();          //bin文件中读取数据，赋给tensor
bool res = GetConstTensorFromBin(PATH+"const_0.bin", weight_tensor,
weight_1_len);

//创建Const类型的权重算子，通过Const算子的属性value，传入tensor
auto conv_weight = op::Const("const_0").set_attr_value(weight_tensor);
```

bool GetConstTensorFromBin(string path, Tensor &weight, uint32_t len）函数表示从bin文件中加载数据：

- path：入参，指定权重文件路径，用于到固定目录例如“../data/weight/”下查找权重文件xx.bin，用户需要自行将权重文件解析为bin文件。（**本实验提供好网络每一层转换好的权重，偏置bin文件**）
- weight：出参，从权重文件中读取的Tensor类型的权重数据。
- len：入参，指定权重数据大小。

5.5.1.4 MaxPool

MaxPool算子定义如下，相关定义解释请见前三个算子：

```
REG_OP(MaxPool)
    .INPUT(x, TensorType({DT_FLOAT16, DT_FLOAT32, DT_DOUBLE, DT_INT8,
                          DT_INT16, DT_INT32, DT_INT64, DT_UINT8,
                          DT_UINT16, DT_QINT8}))
    .OUTPUT(y, TensorType({DT_FLOAT16, DT_FLOAT32, DT_DOUBLE, DT_INT8,
                           DT_INT16, DT_INT32, DT_INT64, DT_UINT8, DT_UINT16,
DT_QINT8}))
    .REQUIRED_ATTR(ksize, ListInt)
    .REQUIRED_ATTR(strides, ListInt)
    .REQUIRED_ATTR(padding, String)
    .ATTR(data_format, String, "NHWC")
    .OP_END_FACTORY_REG(MaxPool)
```

Maxpool算子的代码实现如下：

```
/*
>> 生成Maxpool算子
    pool_name：Maxpool算子的名称
    data：上一个算子。进行构图的连接
*/
Operator GenMaxpoolOp(string pool_name,Operator data){

    auto maxpool = op::MaxPoolV3(pool_name)
        .set_input_x(data)
        .set_attr_strides({1,1,2,2}) // 代表在四个维度（batch、 height,、width、
channels）所移动的步长
        .set_attr_ksize({1,1,2,2}) //代表在四个维度（batch、 height,、width、
channels）池化的尺寸，一般是[1, height, width, 1]
        .set_attr_pads({0,0,0,0})
        .set_attr_data_format("NCHW")
        .set_attr_padding_mode("CALCULATED") //padding_mode默认CALCULATED，三种模式
"SAME" "VALID" or "CALCULATE"
        .set_attr_global_pooling(false)
        .set_attr_ceil_mode(false); //是否在计算输出shape时，使用向上整取，默认false

    TensorDesc tensor_desc(ge::Shape(), FORMAT_NCHW, DT_FLOAT);
    maxpool.update_input_desc_x(tensor_desc);
    maxpool.update_output_desc_y(tensor_desc);
    return maxpool;
}
```

5.5.1.5 Relu

Relu算子的定义如下：

```
REG_OP(Relu)
    .INPUT(x, TensorType({DT_FLOAT, DT_FLOAT16, DT_DOUBLE,
                          DT_INT8, DT_INT32, DT_INT16, DT_INT64,
                          DT_UINT8, DT_UINT16, DT_QINT8}))
    .OUTPUT(y, TensorType({DT_FLOAT, DT_FLOAT16, DT_DOUBLE,
                           DT_INT8, DT_INT32, DT_INT16, DT_INT64,
                           DT_UINT8, DT_UINT16, DT_QINT8}))
    .OP_END_FACTORY_REG(Relu)
```

Relu算子的代码实现如下：

```
/*
>> 生成Relu算子
    relu_name：relu算子的名称
    data：上一个算子。进行构图的连接
*/
Operator GenReluOp(string relu_name,Operator data){

    // 因为relu算子接在bn算子后面，bn算子有多个输出，得指明是data为"y"的输出传入relu，防止因BN有多个输出造成图不明确
    auto relu = op::Relu(relu_name).set_input_x(data, "y");
    TensorDesc tensor_desc(ge::Shape(), FORMAT_ND, DT_FLOAT);
    relu.update_input_desc_x(tensor_desc);
    relu.update_output_desc_y(tensor_desc);
}
```

5.5.1.5 BatchNorm

BatchNorm算子定义如下：

```
REG_OP(BatchNorm)
    .INPUT(x, TensorType({DT_FLOAT16,DT_FLOAT}))
    .INPUT(scale, TensorType({DT_FLOAT}))
    .INPUT(offset, TensorType({DT_FLOAT}))
    .OPTIONAL_INPUT(mean, TensorType({DT_FLOAT}))
    .OPTIONAL_INPUT(variance, TensorType({DT_FLOAT}))
    .OUTPUT(y, TensorType({DT_FLOAT16,DT_FLOAT}))
    .OUTPUT(batch_mean, TensorType({DT_FLOAT}))
    .OUTPUT(batch_variance, TensorType({DT_FLOAT}))
    .OUTPUT(reserve_space_1, TensorType({DT_FLOAT}))
    .OUTPUT(reserve_space_2, TensorType({DT_FLOAT}))
    .OUTPUT(reserve_space_3, TensorType({DT_FLOAT}))
    .ATTR(epsilon, Float, 0.0001)
    .ATTR(data_format, String, "NHWC")
    .ATTR(is_training, Bool, true)
    .OP_END_FACTORY_REG(BatchNorm)
```

BatchNorm算子的代码实现如下：

```
/*
>> 生成Batch Normalization算子
    weight_shape：用于定义BN算子的四个输入权重的shape
```

```
    bn_name：BN算子的名称
    data：上一个算子。进行构图的连接
*/
Operator GenBNOp(Shape weight_shape,string bn_name, Operator data){
    TensorDesc desc_weight_1(weight_shape, FORMAT_NCHW, DT_FLOAT);
    //定义BN算子的四个权重Const算子，分别对应为BN的offset，scale，mean和variance
    Tensor offset_weight_tensor(desc_weight_1);
    Tensor scale_weight_tensor(desc_weight_1);
    Tensor mean_weight_tensor(desc_weight_1);
    Tensor variance_weight_tensor(desc_weight_1);

    uint32_t weight_1_len = weight_shape.GetShapeSize() * sizeof(float);
    //从bin文件中加载BN的offset，offset对应权重文件中的beta，表示输入偏置项
    bool res = GetConstTensorFromBin(kPath+bn_name+".beta.bin",
offset_weight_tensor, weight_1_len);
    if (!res) {
        cout << __LINE__ << "GetConstTensorFromBin Failed!" << endl;
    }
    //从bin文件中加载BN的scale，scale对应权重文件中的gamma，表示输入Scalar
    res = GetConstTensorFromBin(kPath+bn_name+".gamma.bin", scale_weight_tensor,
weight_1_len);
    if (!res) {
        cout << __LINE__ << "GetConstTensorFromBin Failed!" << endl;
    }
    //从bin文件中加载BN的moving_mean，表输入的均值
    res = GetConstTensorFromBin(kPath+bn_name+".moving_mean.bin",
mean_weight_tensor, weight_1_len);
    if (!res) {
        cout << __LINE__ << "GetConstTensorFromBin Failed!" << endl;
    }
    //从bin文件中加载BN的moving_variance，表输入的方差
    res = GetConstTensorFromBin(kPath+bn_name+".moving_variance.bin",
variance_weight_tensor, weight_1_len);
    if (!res) {
        cout << __LINE__ << "GetConstTensorFromBin Failed!" << endl;
    }
    //构造对应的常量算子，用来定义权重
    auto bn_offset = op::Const(bn_name+"_beta")
        .set_attr_value(offset_weight_tensor);
    auto bn_scale = op::Const(bn_name+"_gamma")
        .set_attr_value(scale_weight_tensor);
    auto bn_mean = op::Const(bn_name+"_mean")
        .set_attr_value(mean_weight_tensor);
    auto bn_variance = op::Const(bn_name+"_variance")
        .set_attr_value(variance_weight_tensor);

    //构建bn算子
    auto batchnorm = op::BatchNorm(bn_name)
        .set_input_x(data)
        .set_input_offset(bn_offset)
        .set_input_scale(bn_scale)       //设置输入Scalar
        .set_input_mean(bn_mean)         //设置输入均值
        .set_input_variance(bn_variance) //设置输入方差
        .set_attr_data_format("NCHW")    //设置输入数据的格式NCHW
        .set_attr_is_training(false);    //此时非训练状态，设置成false
```

```
        TensorDesc batchnorm_input_desc_x(ge::Shape(), FORMAT_NCHW, DT_FLOAT);
        TensorDesc batchnorm_output_desc_y(ge::Shape(), FORMAT_NCHW, DT_FLOAT);
        //更新BN的输入信息
        batchnorm.update_input_desc_x(batchnorm_input_desc_x);
        batchnorm.update_input_desc_scale(batchnorm_input_desc_x);
        batchnorm.update_input_desc_offset(batchnorm_input_desc_x);
        batchnorm.update_input_desc_mean(batchnorm_input_desc_x);
        batchnorm.update_input_desc_variance(batchnorm_input_desc_x);
        //更新BN的输出信息
        batchnorm.update_output_desc_y(batchnorm_output_desc_y);
        batchnorm.update_output_desc_batch_mean(batchnorm_output_desc_y);
        batchnorm.update_output_desc_batch_variance(batchnorm_output_desc_y);
}
```

5.5.1.6 Flatten

Flatten算子定义如下：

```
REG_OP(Flatten)
    .INPUT(x, TensorType({DT_INT8, DT_INT16, DT_INT32, DT_INT64,
                          DT_UINT8, DT_UINT16, DT_UINT32, DT_UINT64,
                          DT_FLOAT, DT_FLOAT16}))
    .OUTPUT(y, TensorType({DT_INT8, DT_INT16, DT_INT32, DT_INT64,
                           DT_UINT8, DT_UINT16, DT_UINT32, DT_UINT64,
                           DT_FLOAT, DT_FLOAT16}))
    .ATTR(axis, Int, 1)
    .OP_END_FACTORY_REG(Flatten)
```

Flatten算子的代码实现如下：

```
/*
>> 生成Flatten算子
    flatten_name: Flatten算子的名称
    data: 上一个算子。进行构图的连接
*/
Operator GenFlattenOp(string flatten_name,Operator data){

    //构建Flatten算子
    auto flatten = op::FlattenV2(flatten_name).set_input_x(data);
    //更新算子输入输出信息
    TensorDesc tensor_desc(ge::Shape(), FORMAT_ND, DT_FLOAT);
    flatten.update_input_desc_x(tensor_desc);
    flatten.update_output_desc_y(tensor_desc);
}
```

5.5.1.7 Dense

Dense使用MatMulV2算子实现，MatMulV2算子包含了MatMul和BiasAdd两个部分：

```
REG_OP(MatMulV2)
    .INPUT(x1, TensorType({DT_FLOAT, DT_FLOAT16, DT_INT32, DT_INT8, DT_INT4,
DT_BF16}))
    .INPUT(x2, TensorType({DT_FLOAT, DT_FLOAT16, DT_INT32, DT_INT8, DT_INT4,
DT_BF16}))
    .OPTIONAL_INPUT(bias, TensorType({DT_FLOAT, DT_FLOAT16, DT_INT32, DT_BF16}))
    .OUTPUT(y, TensorType({DT_FLOAT, DT_FLOAT16, DT_INT32, DT_BF16}))
    .OPTIONAL_INPUT(offset_w, TensorType({DT_INT8, DT_INT4}))
    .ATTR(transpose_x1, Bool, false)
    .ATTR(transpose_x2, Bool, false)
    .ATTR(offset_x, Int, 0)
    .OP_END_FACTORY_REG(MatMulV2)
```

说明：

- Dense层的偏置通过OPTIONAL_INPUT的bias输入，bias的值来自bin权重文件
- 属性transpose_x1，transpose_x2，在dense层MatMul的计算逻辑是：MatMul的weight的shape为(out_channels, in_channels), 运算为 $X \cdot W^T$，transpose_x1和transpose_x2分别对应X和W

Dense算子的代码实现如下：

```
/*
>> 生成Dense算子
    input_channel：输入通道数，MatMul的weight的shape为(out_channels, in_channels)
    output_channel：输出通道数
    dense_name：Dense算子的名称
    data：上一个算子，即被flatten后的输入数据，作为MatMul运算中的X。进行构图的连接
*/
Operator GenDenseOp(uint32_t input_channel,uint32_t output_channel,string
dense_name,Operator data){

    // 构造dense层的权重矩阵，权重来自bin文件
    auto matmul_weight_shape = ge::Shape({output_channel, input_channel});
    TensorDesc desc_matmul_weight(matmul_weight_shape, FORMAT_ND, DT_FLOAT);
    Tensor matmul_weight_tensor(desc_matmul_weight);
    uint32_t matmul_weight_len = matmul_weight_shape.GetShapeSize() *
sizeof(float);
    bool res = GetConstTensorFromBin(kPath + dense_name+".weight.bin",
matmul_weight_tensor, matmul_weight_len);
    if (!res) {
        cout << __LINE__ << "GetConstTensorFromBin Failed!" << endl;
    }
    //构造matmul算子的权重常量算子
    auto matmul_weight = op::Const(dense_name+"_weight")
        .set_attr_value(matmul_weight_tensor);

    //构造偏重常量算子，读取偏置参数，作为OPTIONAL_INPUT的bias输入
    auto bias_add_shape = ge::Shape({ output_channel });
    TensorDesc desc_bias_add_const(bias_add_shape, FORMAT_ND, DT_FLOAT);
    Tensor bias_add_const_tensor(desc_bias_add_const);
    uint32_t bias_add_const_len = bias_add_shape.GetShapeSize() * sizeof(float);
    res = GetConstTensorFromBin(kPath + dense_name+".bias.bin",
bias_add_const_tensor, bias_add_const_len);
    if (!res) {
```

```
        cout << __LINE__ << "GetConstTensorFromBin Failed!" << endl;
    }
    auto bias_add_const = op::Const(dense_name+"_bias")
        .set_attr_value(bias_add_const_tensor);

    // 构造MatMulV2算子，三个输入，权重矩阵W，flatten后的输入数据X，偏置bias
    auto matmul = op::MatMulV2(dense_name+"_matmul")
        .set_input_x1(data)
        .set_input_x2(matmul_weight)
        .set_attr_transpose_x2(true)
        .set_input_bias(bias_add_const);

    // 更新算子描述信息
    TensorDesc tensor_desc_matmul(ge::Shape(), FORMAT_ND, DT_FLOAT);
    matmul.update_input_desc_x1(tensor_desc_matmul);
    matmul.update_input_desc_x2(tensor_desc_matmul);
    matmul.update_input_desc_bias(tensor_desc_matmul);
    matmul.update_output_desc_y(tensor_desc_matmul);
}
```

5.5.1.8 Softmax

Softmax使用SoftmaxV2算子，算子定义如下：

```
REG_OP(SoftmaxV2)
    .INPUT(x, TensorType({DT_DOUBLE, DT_FLOAT16, DT_FLOAT}))
    .OUTPUT(y, TensorType({DT_DOUBLE, DT_FLOAT16, DT_FLOAT}))
    .ATTR(axes, ListInt, {-1})
    .OP_END_FACTORY_REG(SoftmaxV2)
```

SoftmaxV2算子的代码实现如下：

```
/*
>> 生成SoftmaxV2算子
    softmax_name：Flatten算子的名称
    data：上一个算子。进行构图的连接
*/
Operator GenSoftmaxOp(string softmax_name, Operator data){
    auto softmax = op::SoftmaxV2(softmax_name).set_input_x(data); //softmax默认
axes为-1
    return softmax;
}
```

5.5.2 构建Graph

5.5.2.1 权重文件转bin文件

我们基于实验四得到flowervgg17的权重文件，权重文件已全部转换好，在(./ckpt)目录下，如下图所示。

/root/exp5/IRBuild/ckpt/*.*

名字	大小	已改变	权限
..		2022/9/9 9:46:33	rwxr-
layer5_conv4.weight.bin	9,216 KB	2022/9/1 9:08:14	rw-r-
layer5_conv3.weight.bin	9,216 KB	2022/9/1 9:08:09	rw-r-
layer5_conv2.weight.bin	9,216 KB	2022/9/1 9:08:04	rw-r-
layer5_conv1.weight.bin	9,216 KB	2022/9/1 9:08:00	rw-r-
layer5_bn4.moving_variance....	2 KB	2022/9/1 9:07:55	rw-r-
layer5_bn4.moving_mean.bin	2 KB	2022/9/1 9:07:54	rw-r-
layer5_bn4.gamma.bin	2 KB	2022/9/1 9:07:54	rw-r-
layer5_bn4.beta.bin	2 KB	2022/9/1 9:07:54	rw-r-
layer5_bn3.moving_variance....	2 KB	2022/9/1 9:07:54	rw-r-
layer5_bn3.moving_mean.bin	2 KB	2022/9/1 9:07:53	rw-r-
layer5_bn3.gamma.bin	2 KB	2022/9/1 9:07:53	rw-r-
layer5_bn3.beta.bin	2 KB	2022/9/1 9:07:52	rw-r-
layer5_bn2.moving_variance....	2 KB	2022/9/1 9:07:52	rw-r-
layer5_bn2.moving_mean.bin	2 KB	2022/9/1 9:07:52	rw-r-
layer5_bn2.gamma.bin	2 KB	2022/9/1 9:07:51	rw-r-
layer5_bn2.beta.bin	2 KB	2022/9/1 9:07:51	rw-r-
layer5_bn1.moving_variance....	2 KB	2022/9/1 9:07:50	rw-r-
layer5_bn1.moving_mean.bin	2 KB	2022/9/1 9:07:50	rw-r-
layer5_bn1.gamma.bin	2 KB	2022/9/1 9:07:50	rw-r-
layer5_bn1.beta.bin	2 KB	2022/9/1 9:07:49	rw-r-

5.5.2.2 从文件读入权重数据

GetConstTensorFromBin函数实现：

```
/*
>>读取bin文件构造tensor，并赋给常量算子Const
    path：指定权重文件路径
    weight：从权重文件中读取的Tensor类型的权重数据
    len：指定权重数据大小
*/
bool GetConstTensorFromBin(string path, Tensor &weight, uint32_t len) {
    //获得文件长度
    ifstream in_file(path.c_str(), std::ios::in | std::ios::binary);
    if (!in_file.is_open()) {
        std::cout << "failed to open" << path.c_str() << '\n';
        return false;
    }
    in_file.seekg(0, ios_base::end); //让文件读指针指向文件结尾
    istream::pos_type file_size = in_file.tellg(); //可以用 tellg 函数获取文件读指针
的位置，此位置即为文件长度
    in_file.seekg(0, ios_base::beg); //让文件读指针指向文件开头

    if (len != file_size) {
        cout << "Invalid Param.len:" << len << " is not equal with binary size（"
<< file_size << "）\n";
        in_file.close();
        return false;
```

```
    }
    char* pdata = new(std::nothrow) char[len];
    if (pdata == nullptr) {
        cout << "Invalid Param.len:" << len << " is not equal with binary size（"
<< file_size << "）\n";
        in_file.close();
        return false;
    }
    //获取文件内容，并赋给Tensor
    in_file.read(reinterpret_cast<char*>(pdata), len);
    auto status = weight.SetData(reinterpret_cast<uint8_t*>(pdata), len);
    if (status != ge::GRAPH_SUCCESS) {
        cout << "Set Tensor Data Failed"<< "\n";
        in_file.close();
        return false;
    }
    in_file.close();
    return true;
}
```

5.5.2.3 定义算子

创建算子实例，关于构建VGG17算子的函数均在5.4.1节中详细阐述，包括算子原型，根据原型设置算子的输入输出及属性。

5.5.2.4 算子构图

基于以上定义好的算子，按照层级堆叠的方式构建VGG17的算子图。

```
//算子构图，这一部分需要学生实现
bool GenGraph(Graph& graph)
{
    auto shape_data = vector<int64_t>({1,3,224,224});//输入数据[N,C,W,H],推理时
batchsize为1
    TensorDesc desc_data(ge::Shape(shape_data), FORMAT_NCHW, DT_FLOAT); //定义算子
信息描述，shape传入desc_data

    // 实例化Data算子，名为data，作为数据输入的算子
    auto data = op::Data("data");
    data.update_input_desc_x(desc_data); //更新data1算子的输入数据信息描述，定义输入数
据的shape，format和dtype
    data.update_output_desc_y(desc_data); //更新data1算子的输出数据信息描述，定义输入数
据的shape，format和dtype

    auto layer1_conv1 = GenConv2dOp(ge::Shape({64,3,3,3}),"layer1_conv1",data);
//卷积算子输入的shape是NCHW
    auto layer1_bn1 = GenBNOp(ge::Shape({64}),"layer1_bn1",layer1_conv1);
    auto layer1_relu1 = GenReluOp("layer1_relu1",layer1_bn1);2
    auto layer1_conv2 =
GenConv2dOp(ge::Shape({64,64,3,3}),"layer1_conv2",layer1_relu1);
    auto layer1_bn2 = GenBNOp(ge::Shape({64}),"layer1_bn2",layer1_conv2);
    auto layer1_relu2 = GenReluOp("layer1_relu2",layer1_bn2);
    auto layer1_maxpool = GenMaxpoolOp("layer1_maxpool",layer1_relu2);
```

```
    /*
    TO-DO:
    1. 参考上述已经搭建好的输入算子和部分VGG17的算子构图部分，请按照VGG17的网络图，同学们自行完成构图的剩余部分的搭建
    注意：要包含最后的softmax层
    */

    // 输入算子，名为data
    std::vector<Operator> inputs{ data };
    // 输出算子，名为pred
    std::vector<Operator> outputs{pred};

    graph.SetInputs(inputs).SetOutputs(outputs);

    return true;
}
```

5.5.3 编译和运行Graph

- 执行./scripts/.testcase300.sh 文件， 成功编译可以得到以下结果

  ```
  bash ./scripts/testcase_300.sh
  ========== Test Start ==========
  Ascend310
  gen
  ========== Generate Graph1 Success! ==========
  Build Model1 SUCCESS!
  Save Offline Model1 SUCCESS!
  run success
  ```

- 生成离线模型。

  在**out**目录下生成离线模型文件**ir_build_vgg17_builtin.om**。

5.5.4 模型推理

利用实验三，加载om离线模型，查看推理结果。

1.在`app.py`中加载离线模型路径

```
if __name__ == '__main__':
    # 加载编译的om离线模型
    engine = ACLEngine('./out/ir_build_vgg17_builtin.om')
    engine.inference('./data/daisy_demo.jpg')
    engine.inference('./data/roses_demo.jpg')
    engine.inference('./data/dandelion_demo.jpg')
    engine.inference('./data/sunflowers_demo.jpg')
    engine.inference('./data/tulips_demo.jpg')
    engine.release_resource()
```

2.执行推理脚本

```
python3.7.5 app.py
```

注:

- 差异性1：实验三的图片预处理中对图片进行标准化，在实验四中并没有，所以将 `./process.py` 中的以下两行注释

  ```
  # 将图片标准化
  # image -= [125.307, 122.961, 113.8575]
  # image /= [51.5865, 50.847, 51.255]
  ```

- 差异性2：实验三的模型是四分类("daisy", "roses", "sunflowers", "tulips")，实验四的模型是五分类("daisy", "dandelion", "roses", "sunflowers", "tulips")，所以将 `./app.py` 中的以下两行进行修改

  ```
  # line8
  CLASSES = ("daisy", "dandelion", "roses", "sunflowers", "tulips")
  # line60
  tuple_st = struct.unpack("5f", bytearray(inference_result[0]))
  ```

得到正确的推理结果应如下图所示：

```
======== inference results: =============
top 0: class:[daisy]: probability:[0.958008]
top 1: class:[tulips]: probability:[0.041779]
top 2: class:[sunflowers]: probability:[0.000069]
top 3: class:[dandelion]: probability:[0.000033]
top 4: class:[roses]: probability:[0.000000]
inference cost time: 9.5ms


======== inference results: =============
top 0: class:[roses]: probability:[1.000000]
top 1: class:[tulips]: probability:[0.000042]
top 2: class:[daisy]: probability:[0.000002]
top 3: class:[dandelion]: probability:[0.000000]
top 4: class:[sunflowers]: probability:[0.000000]
inference cost time: 9.3ms


======== inference results: =============
top 0: class:[dandelion]: probability:[0.995117]
top 1: class:[sunflowers]: probability:[0.002327]
top 2: class:[tulips]: probability:[0.002193]
top 3: class:[daisy]: probability:[0.000521]
top 4: class:[roses]: probability:[0.000002]
inference cost time: 9.3ms


======== inference results: =============
top 0: class:[sunflowers]: probability:[1.000000]
top 1: class:[tulips]: probability:[0.000000]
top 2: class:[roses]: probability:[0.000000]
top 3: class:[dandelion]: probability:[0.000000]
top 4: class:[daisy]: probability:[0.000000]
inference cost time: 9.3ms


======== inference results: =============
top 0: class:[tulips]: probability:[0.998047]
top 1: class:[roses]: probability:[0.001883]
top 2: class:[daisy]: probability:[0.000042]
top 3: class:[sunflowers]: probability:[0.000002]
top 4: class:[dandelion]: probability:[0.000001]
inference cost time: 9.3ms
```

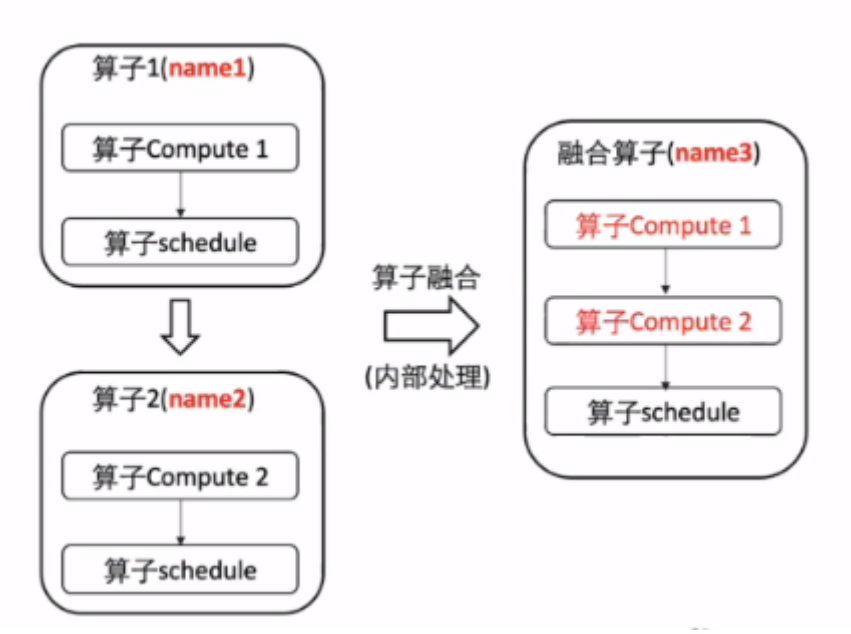
5.6 评分指标

85分：能够成功加载权重数据，成功构图编译运行得到离线.om文件，

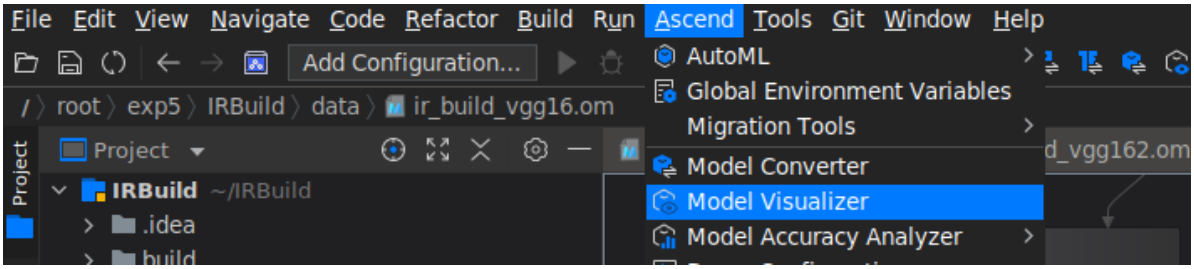
100分：加载离线.om文件，并完成推理并输出指定结果。

5.7 实验思考

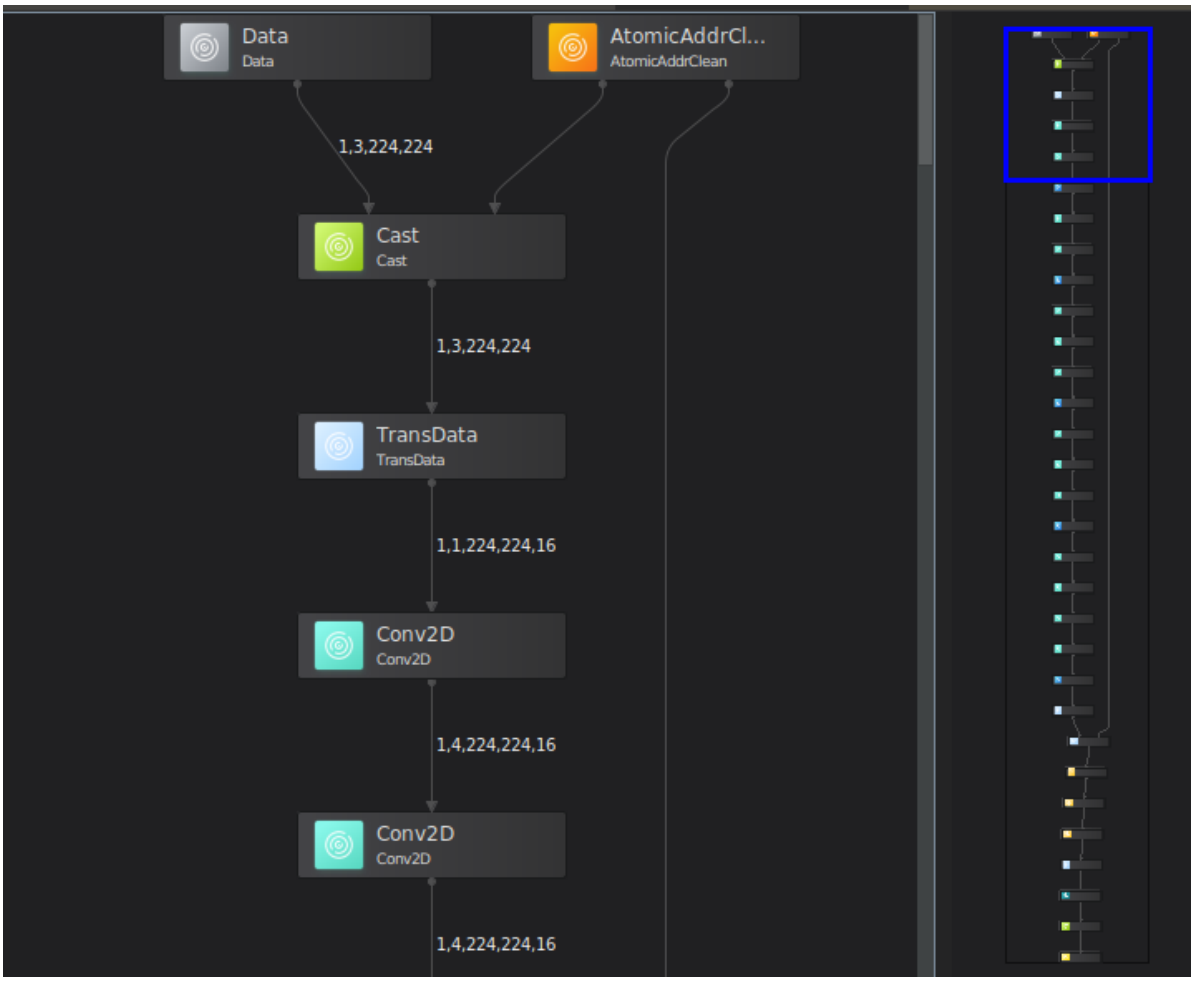
1. 在ATC模型转换的时候会进行算子融合。所谓算子融合即将多个算子的计算逻辑级联成一个算子的计算逻辑（如下图所示），可以节省数据从内存中搬入搬出的时间，优化执行效率，从而加速计算。



试着下载并安装MindStudio，利用菜单栏中的 Ascend-> Model Visualizer 可视化构成的om模型。



可视化的模型部分视图如下，观察有哪些算子融合在一起？



2. 学习视频以及查阅其他相关资料，了解算子融合。